

Práctica evaluable: Simulador de lanzamiento de dados

Resultados de aprendizaje evaluados:

- **RA1.** Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje.
- **RA2.** Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje.

Lenguaje de referencia: Python 3

Librería externa requerida: `rich`

1. Enunciado

Crea una aplicación interactiva de consola en Python que simule el lanzamiento de dados de distintos tipos (D4, D6, D8, D10, D12 y D20). El programa debe permitir al usuario:

- Elegir el tipo de dado que quiere lanzar.
- Indicar cuántos dados quiere lanzar.
- Ver los resultados de forma visual y colorida, usando la librería `rich`.
- Ver una breve animación que simule el proceso de lanzar los dados antes de mostrar el resultado final.

2. Requisitos funcionales y técnicos

El programa debe cumplir, como mínimo, los siguientes requisitos:

1. **Menú principal.** Un bucle que muestre un menú con al menos las opciones “Lanzar dados” y “Salir”, y que se repita hasta que el usuario elija salir.
2. **Selección del tipo de dado.** Mediante una estructura condicional, a partir del número de caras de cada tipo de dado (D4, D6, D8, D10, D12, D20) definido con **constantes**.
3. **Validación de la cantidad de dados.** Debe pedirse por teclado cuántos dados se quieren lanzar, controlando el caso de que el usuario introduzca un texto no numérico, y repitiendo la petición hasta obtener un valor entero positivo válido.
4. **Animación del lanzamiento.** Antes de mostrar el resultado definitivo, el programa debe mostrar varias veces valores aleatorios cambiando rápidamente, simulando que los dados están rodando, usando los componentes de animación de `rich` (por ejemplo, `Live` y `Panel`).
5. **Resultado coloreado.** Cada valor obtenido debe mostrarse con un color distinto según su resultado (por ejemplo, verde si es el valor máximo del dado, rojo si es un 1, amarillo en cualquier otro caso), junto con el total de la tirada, dentro de un `Panel` de `rich`.
6. **Cálculo del promedio.** Además del total, debe calcularse y mostrarse el promedio de los valores obtenidos (total entre cantidad de dados).
7. **Uso de `pass`.** En algún punto del programa debe usarse `pass` como marcador de una futura ampliación todavía no implementada (por ejemplo, una opción de menú “Ver estadísticas” reservada para más adelante).
8. **Control de excepciones adicional.** Cualquier otra entrada del usuario que pueda provocar un error (por ejemplo, una opción de menú no numérica) debe controlarse evitando que el programa tenga una salida inesperada consecuencia de una excepción no controlada.
9. **Documentación del código.** El archivo debe incluir un docstring de módulo que describa su propósito, y comentarios que expliquen los bloques principales del programa.

Restricciones importantes

- No utilizar herramientas del lenguaje de programación que aun no hayamos tratado en clase, como funciones definidas por el propio alumno, listas, tuplas, clases, etc. Por tanto, solo podrás utilizar las herramientas de Python estudiadas en la UT1 y UT2.
- Se tiene que usar la librería `rich` con normalidad, ya que su uso no forma parte de las estructuras propias del lenguaje que se están evaluando.

3. Entrega

1. Crear un proyecto en el IDE de trabajo (por ejemplo, Visual Studio Code), con su propio entorno virtual, e instalar en él la librería `rich` con `pip install rich`.
2. Crear el fichero `requirements.txt` que contenga las dependencias de vuestro entorno virtual.
3. Guardar el programa en un único archivo, por ejemplo `simulador_datos.py`, dentro de ese proyecto.
4. Probar el programa con varios casos de uso distintos, incluyendo al menos un caso límite (por ejemplo, una entrada no numérica al pedir la cantidad de dados, o el valor mínimo posible de dados a lanzar).
5. Realizar el desarrollo utilizando vuestro propio repositorio en GitHub, donde se refleje el historial de cambios desde el inicio del desarrollo y sucesivos commits según habeis ido avanzando en el desarrollo hasta llegar a vuestra solución final.
6. La entrega se realizará a través de la tarea del Aula Virtual, donde debeis:
 - Indicar la url de vuestro repositorio publico en GitHub,
 - Adjuntar vuestra carpeta de proyecto local comprimida en zip o tar, que deberá incluir el fichero `.py` con el código que habeis desarrollado, el fichero `requirements.txt` y la carpeta con el entorno virtual de Python asociado a vuestro proyecto.
 - Adjuntar un video de, como mínimo, tres ejecuciones de prueba distintas que se correspondan con los casos de uso del punto (4) que habeis definido para probar vuestra solución.

3.1. Condiciones relativas a la entrega

1. **Trabajo individual.** La práctica se realiza de forma individual. Queda prohibido compartir el código, total o parcialmente, con otros compañeros o compañeras, así como entregar código copiado de internet o de cualquier otra fuente sin citar.
2. **Desarrollo supervisado en el aula.** El desarrollo y pruebas se realizará en el aula, en horario de clase y bajo supervisión del profesorado, de forma que se pueda observar el proceso de trabajo real del alumnado.
3. **Defensa oral breve.** Cada alumno o alumna deberá defender su programa ante el profesor: explicar brevemente el funcionamiento de dos o tres bloques de código elegidos en el momento, y realizar en directo una pequeña modificación guiada (por ejemplo, cambiar el color de un resultado, añadir un nuevo tipo de dado o modificar un mensaje o cualquier otra que el profesor estime necesaria), para comprobar que comprende lo que ha escrito. Una defensa que no permita verificar la comprensión del código puede suponer la no superación de la práctica, con independencia de la calidad de la solución entregada.
4. **Historial de desarrollo.** Se deberá utilizar un sistema de control de versiones (GitHub) desde el inicio del desarrollo, de forma que el historial de commits permita comprobar la evolución incremental del programa.
5. **Comprobación de similitud entre entregas.** Se compararán las entregas del grupo-clase (mediante una herramienta de detección de similitud de código o de forma manual). Un grado de coincidencia que no pueda justificarse (estructuras y nombres de variable idénticos, mismos comentarios, mismos errores...) invalidará la práctica para todas las personas implicadas, sin

perjuicio de otras medidas disciplinarias que correspondan según la normativa del centro.

6. **Uso de asistentes de IA generativa.** No está permitido el uso de herramientas de inteligencia artificial generativa para escribir el código de la práctica, salvo que el profesorado indique expresamente lo contrario. Ante cualquier duda razonable sobre la autoría del código, prevalecerá el resultado de la defensa oral descrita en el punto 3.

4. Rúbrica de evaluación

Cada resultado de aprendizaje se puntúa sobre el 100 % de forma independiente, repartiendo ese 100 % a partes iguales entre sus criterios de evaluación. Cada criterio se valora en cinco niveles de logro: 0 %, 25 %, 50 %, 75 % y 100 %, de menor a mayor grado de cumplimiento.

4.1. Criterios correspondientes al RA1 (cada criterio: 12,5 %)

Criterio	Descripción	0 %	25 %	50 %	75 %	100 %
a	Se han identificado los bloques que componen la estructura de un programa informático	No se distinguen bloques: el código está desordenado, sin organización clara	Se reconocen algunos bloques, pero mezclados con el resto del código, sin separación real	Los bloques existen pero están mal ordenados	Los bloques están casi todos bien organizados, con algún detalle menor fuera de lugar	El archivo está claramente organizado
b	Se han creado proyectos de desarrollo de aplicaciones	No se ha creado un proyecto: el archivo está suelto, sin estructura de carpetas ni entorno propio	Existe una carpeta, pero sin ninguna organización interna ni entorno	Existe una carpeta de proyecto, pero sin entorno virtual propio o mal configurado	Existe un entorno virtual creado, con algún fallo menor	El programa se entrega dentro de un proyecto bien estructurado, con su propio entorno virtual bien creado y utilizado
c	Se han utilizado entornos integrados de desarrollo	No hay evidencia de uso de un IDE	El archivo se ha abierto en un IDE, pero sin usar ninguna de sus utilidades	Uso básico del IDE, sin aprovechar depurador, terminal integrada ni gestor de paquetes	Uso del IDE con alguna utilidad adicional (terminal integrada o resaltado de errores), pero no todas	Uso completo del IDE: terminal integrada para instalar <i>rich</i> , depurador o resaltado de errores, y gestión del entorno virtual desde el propio IDE

Criterio	Descripción	0 %	25 %	50 %	75 %	100 %
d	Se han identificado los distintos tipos de variables y la utilidad específica de cada uno	Los tipos de dato usados no son adecuados (por ejemplo, comparar texto con números sin convertir)	Se usan tipos básicos, pero con errores frecuentes en su elección	Se usan los tipos básicos correctos, pero sin justificar o aprovechar bien su elección	Casi todos los tipos están bien elegidos, con algún detalle mejorable	Se identifican y usan bien los tipos de datos cada uno donde corresponde
e	Se ha modificado el código de un programa para crear y utilizar variables	No se aprecian variables propias: se repiten valores literales por todo el código	Existen pocas variables; la mayoría de los valores siguen siendo literales sueltos	Se crean variables, pero de forma incompleta o con nombres poco claros	Casi todas las variables relevantes están creadas con nombres claros, salvo alguna excepción	Se crean y usan variables con nombres descriptivos para todos los datos relevantes del programa
f	Se han creado y utilizado constantes y literales. Se han clasificado, reconocido y utilizado en expresiones los operadores del lenguaje	No se usan constantes ni se identifican operadores relevantes	Hay alguna constante aislada, y los operadores se usan de forma incorrecta en varias expresiones	Las constantes están incompletas, o hay algún operador incorrecto puntual	Las constantes están bien definidas y casi todos los operadores son correctos, con algún detalle menor	Se definen constantes y se usan bien operadores aritméticos (+, /), de comparación (==) y lógicos donde corresponde
g	Se ha comprobado el funcionamiento de las conversiones de tipo explícitas e implícitas	No hay conversión de tipos, o se producen errores de tipo no controlados	Hay una conversión explícita, pero con errores	La conversión explícita es correcta, pero no se comprueba ni se explica la conversión implícita	Están presentes ambas conversiones, con alguna imprecisión en la explicación o comprobación	Se usa bien la conversión explícita al leer datos de teclado, y se comprueba y explica la conversión implícita en el cálculo del promedio
h	Se han introducido comentarios en el código	El código no tiene ningún comentario ni docstring	Hay algún comentario aislado, pero no hay docstring de módulo	Los comentarios son insuficientes o poco explicativos, o	Buen nivel de comentarios y docstring presente, con alguna	El archivo incluye un docstring de módulo que explica su propósito, y

Criterio	Descripción	0 %	25 %	50 %	75 %	100 %
				falta el docstring	sección del código sin comentar	comentarios claros en todos los bloques relevantes del programa

4.2. Criterios correspondientes al RA2 (cada criterio: 16,67 %)

Criterio	Descripción	0 %	25 %	50 %	75 %	100 %
a	Se ha escrito y probado código que haga uso de estructuras de selección	No se usa <code>if/elif/else</code> , o se usa de forma incorrecta	Uso muy incompleto, con errores de lógica graves en la selección	Uso incompleto: falta algún caso o la cláusula <code>else</code>	Selección casi completa, con algún caso límite mal resuelto	El menú y la selección del tipo de dado usan bien <code>if/elif/else</code> , cubriendo todos los casos previstos
b	Se han utilizado estructuras de repetición	No se usan bucles, o el programa no repite correctamente el menú o la animación	Hay bucles, pero con fallos graves (por ejemplo, un bucle infinito no intencionado)	Alguno de los bucles no funciona como se espera	Los bucles son casi correctos, con algún detalle menor	Se usan bien un bucle para repetir el menú y bucles para la animación y el lanzamiento de los dados
c	Se han reconocido las posibilidades de las sentencias de salto	No se usa ninguna sentencia de salto	Solo se usa una de las tres sentencias de salto	Se usan dos de las tres, o las tres pero alguna en un lugar sin sentido	Las tres están presentes y casi todas bien usadas, con algún detalle menor	Se usan bien <code>break</code> , <code>continue</code> y <code>pass</code>
d	Se ha escrito código utilizando control de excepciones	No hay ningún bloque <code>try/except</code> ; el programa se detiene con un error no controlado ante una entrada inválida	Hay algún <code>try/except</code> aislado que no llega a evitar errores no controlados	El control de excepciones está presente, pero no cubre todos los casos previstos	Casi todos los casos están cubiertos, salvo algún caso límite sin controlar	Se controlan con <code>try/except</code> todas las entradas de usuario que pueden fallar, sin que el programa se interrumpa nunca de

Criterio	Descripción	0 %	25 %	50 %	75 %	100 %
						forma inesperada
e	Se han creado programas ejecutables utilizando diferentes estructuras de control	El programa no se ejecuta correctamente de principio a fin, o solo usa un tipo de estructura de forma aislada	El programa se ejecuta, pero con errores frecuentes al combinar varias estructuras	El programa se ejecuta, pero la integración entre estructuras es pobre o poco coherente	Buena integración entre estructuras, con algún caso de comportamiento inesperado	El programa es completamente ejecutable y combina de forma coherente selección, repetición, salto y excepciones para resolver el enunciado
f	Se han probado y depurado los programas. Se ha comentado y documentado el código	No se aporta ninguna evidencia de pruebas, y el código no tiene comentarios	Las pruebas son mínimas y los comentarios muy escasos	Las pruebas están incompletas (por ejemplo, sin ningún caso límite) o los comentarios son insuficientes	Buenas pruebas y comentarios, con algún detalle mejorable	Se entregan al menos tres ejecuciones de prueba distintas, incluido algún caso límite, y el código está bien comentado y documentado

5. Cálculo de la nota de la práctica

La nota de cada RA se calcula sumando el porcentaje obtenido en cada uno de sus criterios (según el nivel de logro alcanzado).